Strangler Fig Is Not Enough

Patterns for safe core banking modernization

Wallace Espindola
Solution Architect

wallace.espindola@gmail.com
linkedin.com/in/wallaceespindola | github.com/wallaceespindola

The starting point

- COBOL and CICS on a mainframe post the ledger, run end-of-day batch and feed regulatory reports
- It works, it is audited, and it cannot go offline for a weekend cutover
- Channels want real-time APIs and a better mobile experience
- The people who know the rules are leaving

So the bank modernizes **one capability at a time**, while old and new run side by side.

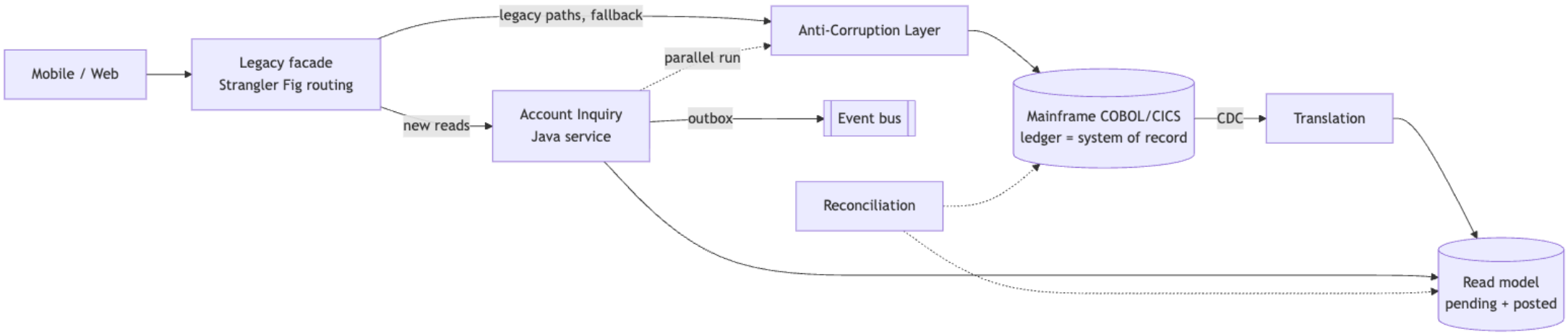
What Strangler Fig gives you

- A **facade** in front of the legacy system
- **Routes** that move one operation at a time to a new service
- A **rollback** that is a routing change, not a deployment

What it does not decide:

- what to extract
- who owns the data while both systems hold a copy
- how to prove the new answer equals the old one

How the patterns fit together



Twelve patterns, twelve jobs

Pattern	Job
Strangler Fig	move traffic one capability at a time
Anti-Corruption Layer	translate records, codes and meanings
Branch by Abstraction	swap a module routing can't isolate
DDD	decide what to extract
CDC + CQRS	feed and scale reads, writes stay on the ledger
Outbox + EDA	publish events reliably
Saga	multi-step flows with compensation, never ledger posting
Parallel Run	prove equivalence before cutover
Hexagonal	keep the domain free of the mainframe adapter
Circuit Breaker + Bulkhead	contain failures

Decide what to extract

- Event Storming and a context map, not the program list
- Capabilities: Account Inquiry, Ledger Posting, Interest, Customer
- **Ledger posting is the core domain**: usually last to move, or retained
- Good first candidates: inquiry, notifications, statements

Avoid one microservice per COBOL program. That reproduces 1980s structure with network latency added.

The anti-corruption layer translates meaning

- EBCDIC to UTF-8, COMP-3 packed decimal to `BigDecimal`
- Return codes and abends to domain errors
- Overloaded fields to explicit types
- "Balance" means ledger, available or memo-posted, depending on context. The ACL makes that explicit.

One ACL per bounded context, versioned contracts, no business logic. Retire it with the legacy capability behind it.

The ACL in code

```
public AccountLookup translate(byte[] raw) {
    LegacyAccountRecord record = LegacyAccountRecord.parse(raw);
    return switch (record.returnCode()) {
        case OK -> new Found(new AccountBalance(record.accountId(), record.balance(), typeOf(record.typeFlag())));
        case NOT_FOUND -> new NotFound(record.accountId());
        case BLOCKED -> new Blocked(record.accountId());
        default -> throw new LegacyTechnicalException("Legacy return code " + record.returnCode());
    };
}

private static BalanceType typeOf(char flag) {
    return switch (flag) {
        case LEDGER_FLAG -> BalanceType.POSTED;
        case MEMO_POSTED_FLAG -> BalanceType.PENDING;
        default -> throw new LegacyTechnicalException("Unknown record type flag " + flag);
    };
}
```

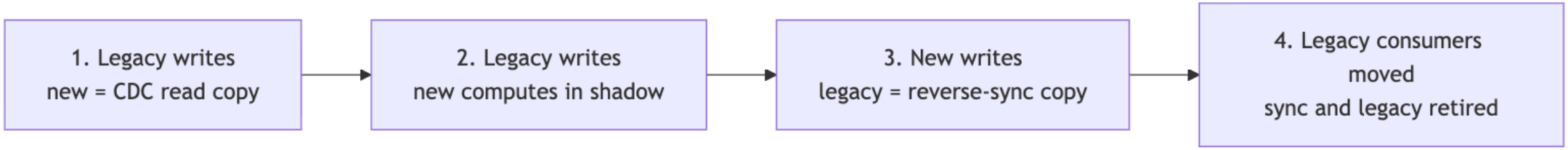
`examples/account-modernization`, package `acl`

Modernize reads first: CDC and CQRS

- CDC feeds a read model while the ledger stays authoritative
- Model **pending vs posted**: cores memo-post during the day, hard-post at end of day
- Lag SLO, per-account ordering, dedupe by log position, replay runbook
- Read-your-own-writes: pending overlay, version token, fallback to legacy

Check CDC feasibility per data store early: Db2 is mature, VSAM and IMS are harder.

Move data ownership in stages



Exactly one writer at every stage. Never dual-write from application code.

Publish events you can trust

- **Transactional outbox**: business change and event row in one local transaction (on the mainframe: same CICS unit of work)
- Relay by polling or log tailing, delivery is at least once
- **Idempotent consumer**: an inbox table keyed by event id
- Partition by account id for ordering; schema registry; DLQ with a runbook
- Keep personal data out of events, or crypto-shred it

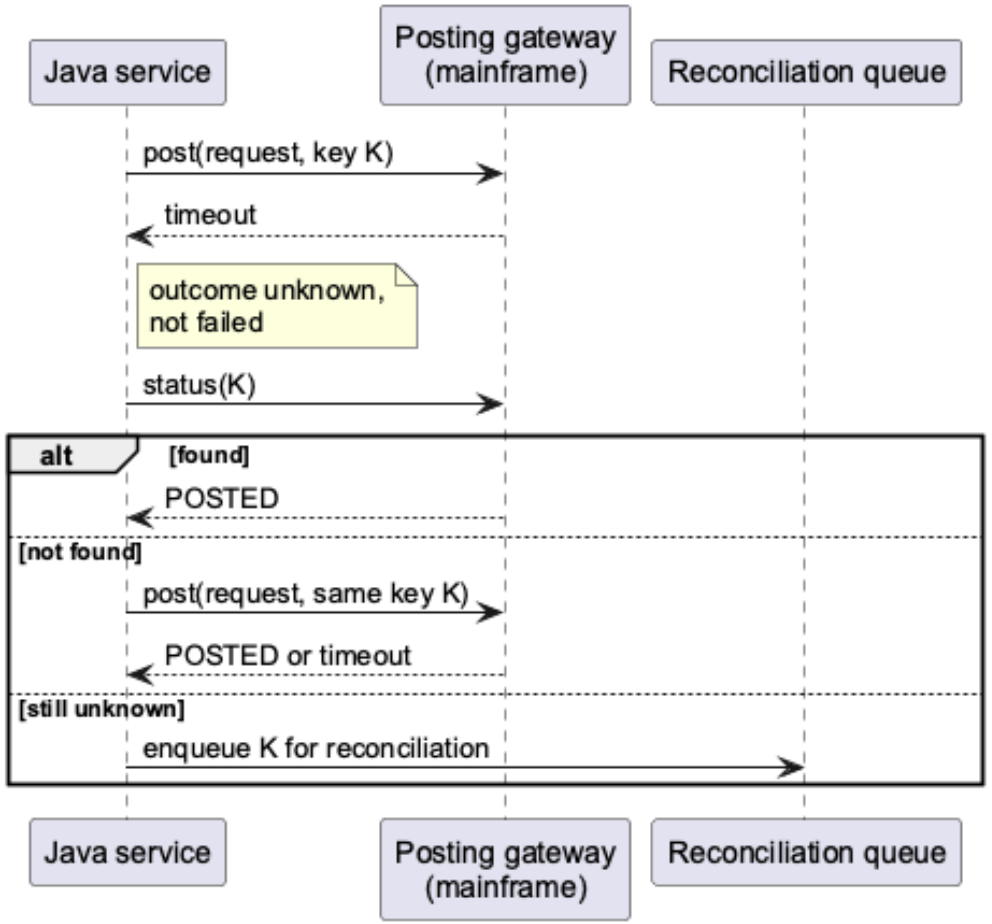
Prove it before you route it

- Run legacy and new on the same inputs, serve legacy, compare
- Tolerance rules, field mapping, known-difference allowlist, triage workflow
- Usual false mismatches: rounding mode, day count, business vs system date
- Shadow side effects off **by configuration**, not by convention

From shadow to decommission



A timeout is an unknown outcome



Unknown outcome in code

```
public PostingResult post(PostingRequest request) {
    try {
        return gateway.post(request);
    } catch (PostingTimeoutException timeout) {
        return resolveUnknownOutcome(request);
    }
}

private PostingResult resolveUnknownOutcome(PostingRequest request) {
    Optional<PostingResult> alreadyPosted = lookup(request);
    if (alreadyPosted.isPresent()) {
        return alreadyPosted.get();
    }
    try {
        return gateway.post(request);
    } catch (PostingTimeoutException timeout) {
        return lookup(request).orElseGet(() -> park(request));
    }
}
```

`examples/account-modernization`, package `posting`

Coordinate multi-step flows with sagas

- Loan origination, onboarding and KYC: orchestration over choreography
- Compensation is a **reversal entry**, not a rollback. Accounting never deletes
- Design the pivot step: irreversible and customer-visible actions after it
- Semantic locks (`PENDING_APPROVAL`) because sagas have no isolation

Not a substitute for atomic ledger posting.

Seven phases, each with an exit criterion

1. **Discover**: contexts agreed, first capability picked, baseline metrics
2. **Encapsulate**: ACL plus facade as pass-through, correlation IDs
3. **Build the read side**: CDC, read model, lag SLO, fallback
4. **Prove**: parallel run, mismatches explained and signed off
5. **Shift reads**: canary to 100%, rollback tested
6. **Extend**: outbox events, replaced calculations
7. **Decommission**: clean reconciliation, audit evidence, sign-off

Anti-patterns to call out early

- Distributed monolith, one service per COBOL program
- Dual writes from application code
- New services reading legacy tables directly
- Ledger first
- Perpetual strangler: no decommission criteria
- Big-bang data migration disguised as incremental
- Leaky ACL: legacy names and codes in new APIs

Where the architect adds value

- Pick the pattern for the problem, avoid pattern-driven overengineering
- Own data ownership, transaction semantics and cutover gates
- Record decisions as ADRs: auditors ask why, years later
- Align with DORA, BCBS 239 and PCI DSS; plan exit strategies
- Model the cost of coexistence: usually the most expensive period
- Decide what stays on the mainframe

Where AI helps, and where it must not decide

Helps: dependency discovery, candidate rule extraction, copybook to schema mapping, characterization test candidates, parallel-run mismatch clustering, documentation recovery.

Guardrails: every rule cites program and line, generated tests are checked against real legacy output, approved deployment models for source code, no line-by-line COBOL-to-Java transliteration.

AI proposes, deterministic tooling verifies, people approve.

Pattern shortcuts

Goal	Patterns
Replace gradually	Strangler Fig + ACL
Modernize reads first	CDC + CQRS
Publish reliable events	Outbox + idempotent consumer
Replace an internal module	Branch by Abstraction + characterization tests
Prove equivalence	Parallel Run + reconciliation
Move write ownership	staged transfer + reverse sync
Safe financial writes	idempotency key + status inquiry

Takeaways

- Strangler Fig moves traffic. It does not move data or prove correctness
- One writer at every stage, and a removal date on every transitional piece
- Model pending vs posted; treat timeouts as unknown outcomes
- Prove with a parallel run, gate with reconciliation
- A capability is done when the new system owns its data, the books balance and the legacy path is off

Thank you

Context, Java examples, diagrams and slides

github.com/wallaceespindola/legacy-modernization-patterns

Wallace Espindola, Solution Architect
linkedin.com/in/wallaceespindola | speakerdeck.com/wallacese